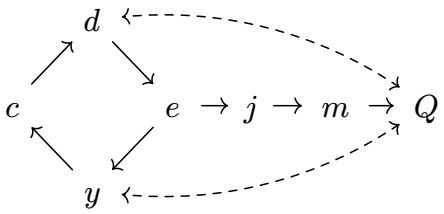
D I Z Z Y

Specification for Event-Driven Architectures with Decoupled Infrastructure

Conrad Mearns Version 0.2.0 — 2026-03-12



Abstract

This document specifies DIZZY, a software architecture framework for building event-driven systems with decoupled infrastructure. DIZZY implements Command Query Responsibility Separation (CQRS) with Event Sourcing using language-agnostic schemas and a code generation pipeline. The architecture enables reversible decisions, supports multiple programming languages, and provides infrastructure independence through strict separation of concerns.

For the philosophy, motivation, and design rationale behind DIZZY, see the companion whitepaper.

Table of Contents

Abstract 1
1 Terminology 4
1a Core Terms 4
1b Architectural Terms 4
1c Implementation Terms 4
2 Architecture Overview 5
2a Architectural Diagram 5
2b Separation of Concerns 6
2c Event Sourcing 6
2d Language-Agnostic Design 6
3 Component Specifications 6
3a Commands 6
3b Events 7
3c Procedures 7
3d Policies 8
3e Projections 8
3f Models 9
3g Queries 9
4 Data Flow Specifications 10
4a Event Loop Flow 10
4b Model Loop Flow 10
4c Message Ordering 10
4d Failure Handling 11
4d1 Procedure Failure 11
4d2 Policy Failure 11
4d3 Projection Failure 11
5 Feature File Specification 11
5a Overview 11
5b models 12
5c queries 12
5d commands 14
5e events 14
5f procedures 15
5g policies 16
5h projections 17
5i Full Example 18
6 Code Generation Pipeline 19
6a Overview 19
6b Directory Structure 20
6c Stage 1 — Author the Feat File 21

6d Stage 2 — Generate Definition Stubs ... 21
6e Stage 3 — Author the Definition Files ... 22
6f Stage 4 — Generate Interfaces and Source Stubs ... 22
6g Regeneration Rules ... 22
6h Import Path Convention ... 23
6i Workflow Summary ... 23
7 Testing Strategy ... 24
7a Architecture: Split Render from Write ... 24
7b Unit Tests — Render Functions ... 24
7c CLI Tests — Typer Commands as Plain Functions ... 24
7d Integration Tests — Snapshot Tests (syrupy) ... 24
7e Test Layout ... 25
8 Implementation Requirements ... 25
8a Naming Conventions ... 25
8b Version Control ... 25
8c Feature Definition Validation ... 26
8d Code Generation Idempotency ... 26
9 Deployment Specifications ... 26
9a Deployment Independence ... 26
9b Wiring Requirements ... 26
9c Deployment Patterns ... 27
9c1 Serverless (AWS Lambda) ... 27
9c2 Message Queue Worker ... 27
9c3 Monolithic Service ... 27
9d Scalability ... 27
10 Security Considerations ... 28
10a Event Store Security ... 28
10b Command Validation ... 28
10c Query Authorization ... 28
10d Procedure Side Effects ... 28
10e Sensitive Data ... 28
10f Deployment Security ... 28
11 References ... 29
11a Normative References ... 29
11b Informative References ... 29
12 Appendix A: Change Log ... 29

1 Terminology

1a Core Terms

Command (c) — An imperative instruction describing an intent to perform work. Commands are ephemeral and trigger exactly one Procedure. Named in `snake_case` present tense (e.g., `ingest_recipe_text`, `start_scan`).

Event (e) — An immutable record of a fact that has occurred in the system. Events are durable and stored permanently in the Event Store. Named in `snake_case` past tense (e.g., `recipe_ingested`, `scan_complete`).

Procedure (d) — A process that handles a Command, performs work (with potential side effects), and emits zero or more Events.

Policy (y) — A process that reacts to an Event, applies business logic, and emits zero or more Commands.

Projection (j) — A process that transforms Events into Model updates, maintaining derived state optimized for queries.

Model (m) — A named database schema providing optimized read access to system state. Models are derived from Events and can be rebuilt from the Event Store at any time.

Query (Q) — A request-response interface for reading data from Models. Queries consist of an input shape (q_i), a processing Protocol (q_p), and an output shape (q_o).

1b Architectural Terms

Event Store — A durable, append-only log containing all Events that have occurred in the system. The Event Store is the single source of truth.

Event Loop — The cycle: Command ⟶ Procedure ⟶ Event ⟶ Policy ⟶ Command.

Model Loop — The cycle: Event ⟶ Projection ⟶ Model ⟶ Query.

Context — An execution environment provided to Procedures, Policies, and Projections containing emitters (callbacks for producing Commands or Events) and, for Procedures, queries (typed interfaces for reading Models).

Feature Definition — A YAML file (`{feature}.feat.yaml`) serving as the single source of truth for a DIZZY feature, declaring all Models, Queries, Commands, Events, Procedures, Policies, and Projections.

1c Implementation Terms

LinkML Schema — A language-agnostic data structure definition using the Linked Data Modeling Language. Schemas define Commands, Events, Models, and Queries.

Pydantic Model — A Python dataclass with type hints and validation logic, generated from LinkML schemas by `linkml gen-pydantic`.

Protocol — A Python `typing.Protocol` specifying the callable signature required for a Procedure, Policy, or Query implementation.

Def — The first generator pass (`dizzy def`). Produces stub `def/` files for human authorship. Never overwrites existing files.

Gen — The second generator pass (`dizzy gen`). Reads authored `def/` files and produces `gen_def/`, `gen_int/`, and `src/` output. Overwrites generated files; never overwrites `src/`.

2 Architecture Overview

2a Architectural Diagram

```
┌──────────────────────────────────────────────────────────────────┐
│                          Event Loop                              │
│                                                                  │
│  Command (c) → Procedure (d) → Event (e) → Policy (y)            │
│        ↑                                          │              │
│        └──────────────────────────────────────────┘              │
└──────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────┐
│                          Model Loop                              │
│                                                                  │
│  Event (e) → Projection (j) → Model (m) → Query (Q)              │
│                                                  │               │
│                                                  ↓               │
│                                       Procedure/Policy           │
└──────────────────────────────────────────────────────────────────┘
```

$$
\begin{tikzcd}
 & d \arrow[dr] \arrow[drr, dashed, bend left] & & & \\
c \arrow[ur] & & e \arrow[r] \arrow[dl] & j \arrow[r] & m \arrow[r] & Q \\
 & y \arrow[ul] \arrow[urr, dashed, bend right] & & &
\end{tikzcd}
$$

The general flow for execution in DIZZY mirrors traditional DDD/EDA. Commands (c) describe an imperative intent. Procedures (d) handle Commands, perform work, and emit Events. Events (e) record immutable facts. Policies (y) react to Events and emit Commands, closing the loop.

Events are the source of truth. To support fast, structured reads, Events are projected onto World Models (m) via Projections (j). Queries (Q) read from Models and are injected into Procedure and Policy contexts. Since $|e|$ is decoupled from $|m|$, Projections serve as the mapping function between the two.

2b Separation of Concerns

DIZZY enforces strict separation between:

Write Side (CQRS Command Side):
- Commands express intent
- Procedures perform work
- Events record facts
- All writes flow through the Event Store

Read Side (CQRS Query Side):
- Events trigger Projections
- Projections update Models
- Queries read from Models
- Multiple Models can represent the same data differently

2c Event Sourcing

The system stores the complete sequence of Events rather than current state. This enables:

1. Complete audit trail
2. Time travel (reconstruct any historical state)
3. Debug by replay
4. Multiple model representations
5. Late schema binding — add new Projections to existing Events at any time

2d Language-Agnostic Design

Components communicate exclusively through serialized data structures:

- No shared memory
- No library dependencies between components
- Each component can be implemented in any language
- Type safety maintained through schema definitions

3 Component Specifications

3a Commands

Commands MUST be:
- **Imperative**: Express intent
- **Ephemeral**: Not persisted long-term
- **Unique**: Identified by a unique `command_id`

Commands SHOULD:
- Use `snake_case` present tense naming
- Include all parameters needed for execution
- Be idempotent when possible

Lifecycle:
1. Command is created (by user, policy, or external system)
2. Command is validated against schema
3. Command is routed to exactly one Procedure
4. Command is processed
5. Command is discarded (not stored long-term)

3b Events

Events MUST be:
- **Immutable**: Never modified after creation
- **Durable**: Stored permanently in Event Store
- **Factual**: Describe what happened (past tense)
- **Unique**: Identified by a unique `event_id`

Events SHOULD:
- Use `snake_case` past tense naming
- Include timestamp
- Include sufficient context for Projections
- Be granular (one fact per event)

Lifecycle:
1. Event is emitted by a Procedure
2. Event is validated against schema
3. Event is appended to Event Store
4. Event triggers zero or more Policies
5. Event triggers zero or more Projections
6. Event is retained indefinitely

3c Procedures

Procedures MUST:
- Accept exactly one Command type as input
- Receive a Context with emitters and queries
- Return nothing (use emitters for output)
- Be deterministic given the same inputs and query results

Procedures MAY:
- Perform side effects (I/O, external API calls)
- Read from Models via queries
- Emit multiple Events
- Emit different Events based on execution results

Procedures SHOULD:
- Emit error events rather than raising exceptions for domain errors
- Only raise exceptions for infrastructure failures

Context structure:

```
@dataclass
class procedure_name_emitters:
    event_name: Callable[[event_name], None]

@dataclass
class procedure_name_queries:
    query_name: query_name_query

@dataclass
class procedure_name_context:
    emit: procedure_name_emitters
    query: procedure_name_queries
```

3d Policies

Policies MUST:

- React to exactly one Event type
- Receive a Context with emitters only
- Return nothing (use emitters for output)

Policies MAY:

- Emit zero or more Commands
- Implement complex business logic
- Emit different Commands based on the Event content

Policies SHOULD:

- Encode business rules, not technical operations
- Avoid complex state management

Context structure (emitters only — Policies do not receive query access):

```
@dataclass
class policy_name_emitters:
    command_name: Callable[[command_name], None]

@dataclass
class policy_name_context:
    emit: policy_name_emitters
```

For policies with no `emits`, the emitters dataclass has `pass`.

3e Projections

Projections MUST:

- Be idempotent (replay safe)
- Be deterministic (same Event produces same Model update)
- Handle Event ordering issues gracefully
- Accept an Event and a Context holding SQLAlchemy sessions for the model schemas they write

Projections MAY:

- Write to multiple Model schemas per Event

Projections MUST support full Model rebuild:

1. Clear existing Model
2. Replay all Events from Event Store
3. Apply each Event to Model
4. Resulting Model MUST match production Model

Context and Protocol structure:

```
@dataclass
class projection_name_context:
    """SQLAlchemy sessions for schemas written by this projection."""
    schema_name: Any  # SQLAlchemy session for the schema_name schema

class projection_name_projection(Protocol):
    """Description of what this projection does."""

    def __call__(
        self, event: event_name, context: projection_name_context
    ) -> None:
        """Apply the projection — mutate model state in response to the
event."""
        ...
```

3f Models

Models:

- Represent derived state (not the source of truth)
- Optimize specific query patterns
- Can be rebuilt from Events at any time
- MAY use any database technology

Models MUST:

- Be eventually consistent with the Event Store
- Support concurrent updates if multiple instances exist
- Handle duplicate Events idempotently

3g Queries

Queries MUST:

- Read from Models (not the Event Store)
- Be side-effect free
- Define typed inputs and outputs via LinkML schemas

Queries MAY:

- Join multiple tables within a single Model schema

- Implement caching

Queries consist of three components:

1. **Query Input** (q_i): A LinkML-defined data shape for input parameters
2. **Query Process** (q_p): A Protocol for the callable that accepts input and a context
3. **Query Output** (q_o): A LinkML-defined data shape for the return value

Each query is bound to exactly one Model schema. The generated context holds a SQLAlchemy session for that schema.

4 Data Flow Specifications

4a Event Loop Flow

1. Command arrives (from user, policy, or external system)
2. System validates Command against schema
3. System routes Command to registered Procedure
4. Procedure executes with provided Context
5. Procedure emits Events via `context.emit`
6. Events are validated and appended to Event Store
7. Events trigger registered Policies
8. Policies execute and emit new Commands
9. Loop continues from step 1

4b Model Loop Flow

1. Event is appended to Event Store
2. Event triggers registered Projections
3. Projections update Models via SQLAlchemy sessions
4. Models become available for Queries
5. Procedures use Queries via `context.query`
6. Query results influence future Commands/Events

4c Message Ordering

Events MUST be:

- Totally ordered within a single Event Store
- Timestamped with a monotonic clock
- Sequentially numbered

The system SHOULD:

- Process Events in order when possible
- Handle out-of-order Events gracefully in Projections
- Use correlation IDs to track related Events

4d Failure Handling

4d1 Procedure Failure

If a Procedure fails:

1. Command is NOT marked as processed
2. Procedure MAY emit an error Event before failing
3. System MAY retry Command (deployment-specific)
4. No partial Events are committed

4d2 Policy Failure

If a Policy fails:

1. Event remains in Event Store
2. Policy MAY be retried (deployment-specific)
3. Other Policies still process the Event
4. No partial Commands are emitted

4d3 Projection Failure

If a Projection fails:

1. Event remains in Event Store
2. Model MAY be inconsistent temporarily
3. Projection SHOULD be rerun from last checkpoint
4. Other Projections continue processing

5 Feature File Specification

5a Overview

A `.feat.yaml` file is the primary authoring surface in DIZZY. It describes a single feature's domain model and processing logic at a high level of abstraction — without specifying databases, frameworks, or infrastructure.

The generator pipeline reads a `.feat.yaml` and produces:

- LinkML schema stubs (`def/`) for each section requiring human authorship
- Generated Python models and interfaces (`gen_def/`, `gen_int/`)
- Implementation stubs (`src/`) for the developer to fill in

Top-level structure:

```
description: <string>   # Human-readable description of the feature (optional)

models:      <map>       # Domain value objects / data shapes
queries:     <map>       # Read interfaces (input + output)
commands:    <map>       # Write intents
events:      <map>       # Immutable facts (what happened)
procedures:  <map>       # Command handlers (do work, emit events)
```

```yaml
policies:    <map>       # Event handlers (react, issue commands)
projections: <map>       # Read-model builders (event → queryable state)
```

All sections are optional. The generator skips sections not present.

5b models

Named database schemas. Each entry represents a logical grouping of related classes (tables) for a single database. The feat file declares schema names and optional descriptions only. Actual class definitions live in the corresponding `def/models/<schema_name>.yaml` LinkML file, which is authored separately and may grow without touching the feat file.

```yaml
models:
  recipes: Full recipe database — recipes, steps, and ingredients
```

`def/models/recipes.yaml` (hand-authored) then defines all classes:

```yaml
classes:
  Recipe:
    attributes:
      title: ...
  Step:
    attributes:
      body: ...
  Ingredient:
    attributes:
      name: ...
      quantity: ...
```

Use plural, lowercase (snake_case) names.

`dizzy def` generates (stub, never overwritten):

- `def/models/<schema_name>.yaml` — stub LinkML schema

Gen generates (by running the LinkML toolchain):

- `gen_def/pydantic/models/<schema_name>.py` — Pydantic models (via `linkml gen-pydantic`)
- `gen_def/sqla/models/<schema_name>.py` — SQLAlchemy models (via `linkml gen-sqla`)

5c queries

Named read operations. Each query declares the single Model schema it reads from. IO types are defined in authored LinkML stubs and fleshed out at implementation time.

```yaml
queries:
  get_recipe_text:
    description: Retrieves raw recipe text given a source reference
    model: recipes
  get_recipe:
    description: Retrieves a structured recipe by ID
    model: recipes
```

Fields:

- `description` (required): what this query does
- `model` (required): the schema name from `models` that this query reads from

`dizzy def` generates (stub, never overwritten):

- `def/queries/<query_name>.yaml` — single LinkML stub containing `<QueryName>Input` and `<QueryName>Output` class stubs

Example `def/queries/get_recipe_text.yaml`:

```yaml
id: https://example.org/queries/get_recipe_text
name: get_recipe_text
description: Retrieves raw recipe text given a source reference
prefixes:
  linkml: https://w3id.org/linkml/
default_range: string
imports:
  - linkml:types
classes:
  GetRecipeTextInput:
    description: Input for get_recipe_text
    attributes: {}
  GetRecipeTextOutput:
    description: Output for get_recipe_text
    attributes: {}
```

Gen generates:

- `gen_def/pydantic/query/<query_name>.py` — Pydantic models for both Input and Output
- `gen_int/python/query/<query_name>.py` — `QueryProcess` Protocol + context dataclass:

```python
# AUTO-GENERATED — do not edit
from dataclasses import dataclass
from typing import Protocol, Any

from gen_def.pydantic.query.get_recipe_text import (
    GetRecipeTextInput,
    GetRecipeTextOutput,
)


@dataclass
class get_recipe_text_context:
    """SQLAlchemy session for the schema read by this query."""
    recipes: Any  # SQLAlchemy session for the recipes schema


class get_recipe_text_query(Protocol):
    """Retrieves raw recipe text given a source reference"""

    def __call__(
```

```
        self, input: GetRecipeTextInput, context: get_recipe_text_context
    ) -> GetRecipeTextOutput:
        ...
```

Queries declared in a Procedure's `queries:` list are injected into that Procedure's `_queries` context dataclass as typed fields.

5d commands

Named write intents. Value is either a short description string, or a map with description and optional attributes.

```yaml
commands:
  ingest_recipe_text: Initiates ingestion of a recipe from a raw text source

  upload_blob_using_manifest:
    description: Uploads a blob using manifest information
    attributes:
      manifest_id:
        type: string
        required: true
```

`dizzy def` generates (stub, never overwritten):

- `def/commands.yaml` — LinkML stub listing all commands with attributes

Gen generates:

- `gen_def/pydantic/commands.py` — Pydantic models for all commands

5e events

Immutable domain facts. Value is either a description string, or a map with description and optional attributes.

```yaml
events:
  recipe_ingested: A recipe was successfully ingested and validated

  scan_item_found:
    description: Found a file while scanning
    attributes:
      file_path:
        type: string
        required: true
      file_hash:
        type: string
```

`dizzy def` generates (stub, never overwritten):

- `def/events.yaml` — LinkML stub listing all events with attributes

Gen generates:

- `gen_def/pydantic/events.py` — Pydantic models for all events

5f procedures

Command handlers. Each procedure is bound to one command, declares queries it uses, and events it may emit.

```yaml
procedures:
  extract_and_transform_recipe:
    description: >
      Queries raw recipe text via source_ref, then uses an LLM to extract a
structured
      recipe, validated against the recipe model schema.
    command: ingest_recipe_text
    queries:
      - get_recipe_text
    emits:
      - recipe_ingested
```

Fields:

- `command` (required): the command this procedure handles
- `queries` (optional): list of query names this procedure needs access to
- `emits` (optional): list of event names this procedure may emit

Gen generates:

- `gen_int/python/procedure/<procedure_name>_context.py` — context dataclass with `emit` and `query` nested dataclasses:

```python
# AUTO-GENERATED — do not edit
from dataclasses import dataclass
from typing import Callable

from gen_def.pydantic.events import recipe_ingested
from gen_int.python.query.get_recipe_text import get_recipe_text_query


@dataclass
class extract_and_transform_recipe_emitters:
    recipe_ingested: Callable[[recipe_ingested], None]


@dataclass
class extract_and_transform_recipe_queries:
    get_recipe_text: get_recipe_text_query


@dataclass
class extract_and_transform_recipe_context:
    emit: extract_and_transform_recipe_emitters
    query: extract_and_transform_recipe_queries
```

- `gen_int/python/procedure/<procedure_name>_protocol.py` — Protocol stub:

```python
# AUTO-GENERATED — do not edit
from typing import Protocol

from gen_def.pydantic.commands import ingest_recipe_text
from gen_int.python.procedure.extract_and_transform_recipe_context import (
    extract_and_transform_recipe_context,
)


class extract_and_transform_recipe_protocol(Protocol):
    """Queries raw recipe text via source_ref, then uses an LLM to extract..."""

    def __call__(
        self,
        context: extract_and_transform_recipe_context,
        command: ingest_recipe_text,
    ) -> None:
        ...
```

- `src/procedure/<procedure_name>.py` — empty implementation stub (skipped if already exists)

5g policies

Event-driven reaction handlers. Each policy listens to one event and optionally emits commands. Policies do not receive query access.

```yaml
policies:
  trigger_priority_manifest:
    description: Issues command to create image priority manifest when scan
completes
    event: scan_complete
    emits:
      - create_image_priority_manifest
```

Fields:

- `event` (required): the event that triggers this policy
- `emits` (optional): list of command names this policy may dispatch

Gen generates:

- `gen_int/python/policy/<policy_name>_context.py` — context dataclass with emitters only:

```python
# AUTO-GENERATED — do not edit
from dataclasses import dataclass
from typing import Callable

from gen_def.pydantic.commands import create_image_priority_manifest
```

```
@dataclass
class trigger_priority_manifest_emitters:
    create_image_priority_manifest: Callable[[create_image_priority_manifest],
None]


@dataclass
class trigger_priority_manifest_context:
    emit: trigger_priority_manifest_emitters
```

- `gen_int/python/policy/<policy_name>_protocol.py` — Protocol stub:

```
# AUTO-GENERATED — do not edit
from typing import Protocol

from gen_def.pydantic.events import scan_complete
from gen_int.python.policy.trigger_priority_manifest_context import (
    trigger_priority_manifest_context,
)


class trigger_priority_manifest_protocol(Protocol):
    """Issues command to create image priority manifest when scan completes"""

    def __call__(
        self, event: scan_complete, context: trigger_priority_manifest_context
    ) -> None:
        ...
```

- `src/policy/<policy_name>.py` — implementation stub (skipped if already exists)

5h projections

Build queryable read models in response to a single event. Each projection listens to exactly one event and writes into one or more model schemas.

```
projections:
  recipe_library:
    description: Adds ingested recipe to the recipe library
    event: recipe_ingested
    models:
      - recipes
```

Fields:

- `description` (required): what this projection does
- `event` (required): the single event that triggers this projection
- `models` (required): list of schema names from `models` that this projection writes into

Gen generates `gen_int/python/projection/<projection_name>_projection.py` — a combined context dataclass and Protocol stub:

```python
# AUTO-GENERATED — do not edit
from dataclasses import dataclass
from typing import Protocol, Any

from gen_def.pydantic.events import recipe_ingested


@dataclass
class recipe_library_context:
    """SQLAlchemy sessions for schemas written by this projection."""
    recipes: Any  # SQLAlchemy session for the recipes schema


class recipe_library_projection(Protocol):
    """Adds ingested recipe to the recipe library"""

    def __call__(
        self, event: recipe_ingested, context: recipe_library_context
    ) -> None:
        """Apply the projection — mutate model state in response to the
event."""
        ...
```

The `Any` session type is a placeholder; the implementor binds it to a concrete SQLAlchemy `Session` when wiring up the projection.

- `src/projection/<projection_name>.py` — implementation stub (skipped if already exists)

5i Full Example

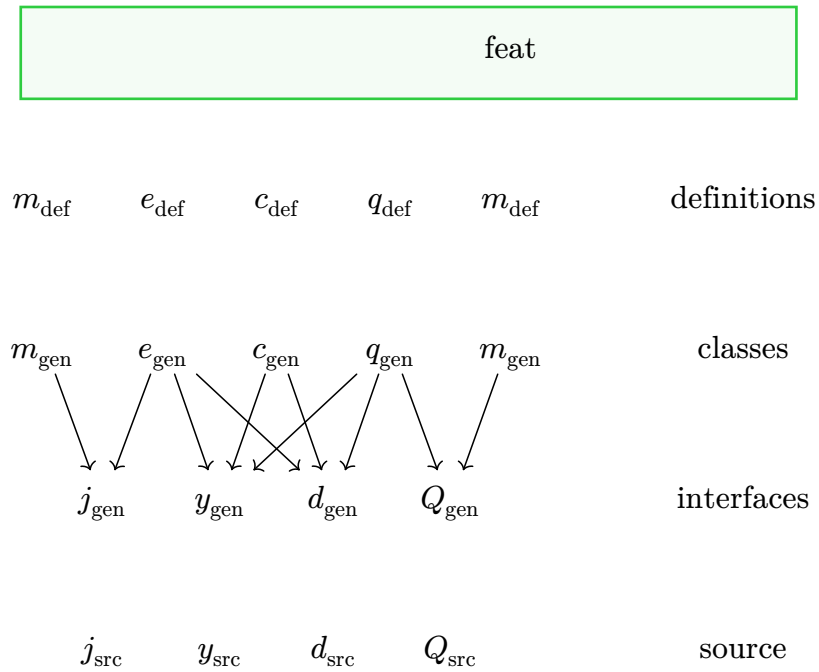
```yaml
description: Recipe App

models:
  recipes: Full recipe database — recipes, steps, and ingredients

queries:
  get_recipe_text:
    description: Retrieves raw recipe text given a source reference
    model: recipes
  get_recipe:
    description: Retrieves a structured recipe by ID
    model: recipes

commands:
  ingest_recipe_text: Initiates ingestion of a recipe from a raw text source
```

```yaml
events:
  recipe_ingested:
    description: A recipe was successfully extracted and validated
    attributes:
      recipe_id:
        type: string
        required: true
      source_ref:
        type: string
        required: true

procedures:
  extract_and_transform_recipe:
    description: >
      Queries raw recipe text via source_ref, then uses an LLM to extract a
structured
      recipe (title, ingredients, steps, time, cost), validated against the
recipe model.
    command: ingest_recipe_text
    queries:
      - get_recipe_text
    emits:
      - recipe_ingested

policies:
  index_recipe_on_ingest:
    description: Adds recipe to the library projection when ingested
    event: recipe_ingested

projections:
  recipe_library:
    description: Adds ingested recipe to the recipe library
    event: recipe_ingested
    models:
      - recipes
```

6 Code Generation Pipeline

6a Overview

DIZZY uses a two-step, multi-stage code generation pipeline to maintain type safety while preserving human authorship over data shapes.



The split between `dizzy def` and `dizzy gen` exists because `def/` files require human authorship between the two passes — they cannot be fully derived from the feat file alone.

6b Directory Structure

Given a feature at `app/my_feature/my_feature.feat.yaml`, the full output layout is:

```
app/my_feature/
  def/                          # hand-authored LinkML stubs (never overwritten)
    models/
      recipes.yaml
    queries/
      get_recipe_text.yaml
      get_recipe.yaml
    commands.yaml
    events.yaml
  gen_def/                      # generated type definitions (overwritten by
gen)
    pydantic/
      models/
        recipes.py
      query/
        get_recipe_text.py
        get_recipe.py
      commands.py
      events.py
    sqla/
      models/
        recipes.py
  gen_int/                      # generated interfaces (overwritten by gen)
```

```
    python/
      query/
        get_recipe_text.py
        get_recipe.py
      procedure/
        extract_and_transform_recipe_context.py
        extract_and_transform_recipe_protocol.py
      policy/
        index_recipe_on_ingest_protocol.py
      projection/
        recipe_library_projection.py
  src/                          # implementation stubs (only created, never
overwritten)
    query/
      get_recipe_text.py
      get_recipe.py
    procedure/
      extract_and_transform_recipe.py
    policy/
      index_recipe_on_ingest.py
    projection/
      recipe_library.py
```

`dizzy gen` emits an empty `__init__.py` in every generated directory so the output tree is a valid Python package and root-relative imports resolve correctly.

Sections with no content in the feat file produce no output.

6c Stage 1 — Author the Feat File

Write `my_feature.feat.yaml` by hand. Declare models, commands, events, procedures, policies, and projections at the intent level. No types, no schemas, no implementation details yet.

6d Stage 2 — Generate Definition Stubs

```
dizzy def <feat_file> <output_dir>
```

Reads the feat file and generates empty `def/` stub files for everything that requires human schema authorship:

- `def/models/<schema_name>.yaml` — stub LinkML schema per model (skipped if already exists)
- `def/queries/<query_name>.yaml` — stub with `<QueryName>Input` and `<QueryName>Output` class stubs (skipped if already exists)
- `def/commands.yaml` — stub listing all commands (skipped if already exists)
- `def/events.yaml` — stub listing all events (skipped if already exists)

After running, Dizzy prints:

```
Generated def/ stubs. Next steps:
  1. Fill in class definitions in def/models/*.yaml
  2. Add input/output shapes in def/queries/*.yaml
```

```
  3. Add attributes to def/commands.yaml and def/events.yaml
  4. Run: dizzy gen <feat_file> <output_dir>
```

6e Stage 3 — Author the Definition Files

Edit the generated `def/` stubs:

- Add classes, attributes, and relationships to each model schema
- Add typed attributes to commands and events as needed
- Add input/output shapes to query stubs

These files belong to the developer — Dizzy will never overwrite them.

6f Stage 4 — Generate Interfaces and Source Stubs

```
dizzy gen <feat_file> <output_dir>
```

Reads both the feat file and the authored `def/` files, then generates:

`gen_def/` — produced by running `linkml gen-pydantic` (and `linkml gen-sqla` for models) on the authored `def/` schemas.

`gen_int/` — Protocol stubs derived from the feat file structure (queries, procedures, policies, projections), using the `gen_def/` types as references in imports.

`src/` — implementation stubs, one per interface. Each stub imports its Protocol and raises `NotImplementedError`. Source stubs are only created if the file does not already exist, so gen is safe to re-run.

After running, Dizzy prints a per-section summary and:

```
Generated interfaces and source stubs. Next steps:
  Implement the src/ files to complete your feature.
```

6g Regeneration Rules

The following are safe to regenerate freely:

- All files under `gen_def/` and `gen_int/` — overwritten on every `dizzy gen` run
- Adding new Commands, Events, Procedures, Policies, Projections, or Queries

The following require manual updates after regeneration:

- Schema changes affecting existing implementations
- Adding new attributes to Commands or Events
- Changing Procedure or Policy signatures

The generator MUST NEVER modify:

- Files in `src/` (implementation stubs once created)
- Files in `def/` (hand-authored schemas)
- Tests
- Documentation

6h Import Path Convention

All generated files use **relative Python imports**. The intent is that the entire output directory is portable — it can be copied or symlinked into any project structure without changing import paths. The feature output directory must be on `sys.path` for imports to resolve at runtime.

From	Importing	Import
`gen_int/python/query/`	Query input/output models	`from gen_def.pydantic.query.<name> import ...`
`gen_int/python/procedure/`	Pydantic events	`from gen_def.pydantic.events import ...`
`gen_int/python/procedure/`	Pydantic commands	`from gen_def.pydantic.commands import ...`
`gen_int/python/procedure/`	Query Protocols	`from gen_int.python.query.<name> import ...`
`gen_int/python/policy/`	Pydantic events	`from gen_def.pydantic.events import ...`
`gen_int/python/policy/`	Pydantic commands	`from gen_def.pydantic.commands import ...`
`gen_int/python/projection/`	Pydantic events	`from gen_def.pydantic.events import ...`
`src/procedure/`	Procedure Protocol + context	`from gen_int.python.procedure.<name>_pr import ...`
`src/policy/`	Policy Protocol	`from gen_int.python.policy.<name>_proto import ...`
`src/projection/`	Projection Protocol + context	`from gen_int.python.projection.<name>_p import ...`

6i Workflow Summary

Step	Command	You do next
1	—	Write `my_feature.feat.yaml`
2	`dizzy def`	Edit `def/` schema stubs
3	—	Author class definitions and attributes
4	`dizzy gen`	Implement `src/` stubs

7 Testing Strategy

7a Architecture: Split Render from Write

Every generator module exposes two layers:

1. **`render_*(feat, ...) -> str`** — pure function, takes parsed feat data and returns the file content as a string. No filesystem access. Fully unit-testable in isolation.
2. **`write_*(feat, output_dir, ...)`** — thin wrapper that calls `render_*` and writes the result to the correct path under `output_dir`. This layer is covered by integration tests.

This split means the majority of tests never touch the filesystem and run fast.

7b Unit Tests — Render Functions

Each `render_*` function gets direct unit tests asserting on the returned string. Tests live in `dizzy/tests/generators/test_<section>.py`. A representative feat fixture is defined once in `dizzy/tests/conftest.py` and shared across all generator tests.

```
def test_render_procedure_context(recipe_feat):
    result = render_procedure_context("extract_and_transform_recipe",
recipe_feat)
    assert "class extract_and_transform_recipe_context" in result
    assert "get_recipe_text: get_recipe_text_query" in result
```

7c CLI Tests — Typer Commands as Plain Functions

Typer commands are plain Python functions. Tests call them directly without subprocess or `CliRunner`. The `tmp_path` pytest fixture provides the output directory.

```
from dizzy.cli import def_cmd

def test_def_creates_def_stubs(tmp_path: Path) -> None:
    def_cmd(feat_file=FIXTURES_DIR / "recipe.feat.yaml", output_dir=tmp_path)
    assert (tmp_path / "def" / "commands.yaml").exists()
    assert (tmp_path / "def" / "events.yaml").exists()
```

7d Integration Tests — Snapshot Tests (syrupy)

End-to-end tests call the full generator pipeline against a known feat fixture, write output to a `tmp_path` directory, and compare every generated file against a saved snapshot.

Snapshots live in `dizzy/tests/snapshots/` and are committed to version control. They serve as living documentation of exactly what each generator produces.

```
def test_gen_full_example(tmp_path, snapshot):
    feat = load_feat("tests/fixtures/recipe.feat.yaml")
    gen(feat, tmp_path)
    for path in sorted(tmp_path.rglob("*.py")):
        assert path.read_text() ==
snapshot(name=str(path.relative_to(tmp_path)))
```

To update snapshots after an intentional template change:

```
pytest --snapshot-update
```

7e Test Layout

```
dizzy/
  tests/
    conftest.py               # shared feat fixtures
    fixtures/
      recipe.feat.yaml        # full example feat used across tests
    snapshots/                # syrupy snapshot files (committed)
    generators/
      test_models.py
      test_commands.py
      test_events.py
      test_queries.py
      test_procedures.py
      test_policies.py
      test_projections.py
    test_cli.py               # end-to-end def + gen integration tests
```

8 Implementation Requirements

8a Naming Conventions

Concept	Convention	Examples
Commands	`snake_case` present tense	`ingest_recipe_text`, `start_scan`
Events	`snake_case` past tense	`recipe_ingested`, `scan_complete`
Procedures	`snake_case` verb phrase	`extract_and_transform_recipe`, `partition_scan`
Policies	`snake_case` rule description	`index_recipe_on_ingest`, `hash_priority_files`
Model schemas	plural `snake_case` nouns	`recipes`, `scan_results`
Queries	`snake_case` starting with `get_`, `find_`, or `list_`	`get_recipe_text`, `list_active_users`
Projections	`snake_case` noun phrase	`recipe_library`, `scan_result`

8b Version Control

Projects MUST version control:

- Feature Definitions (`*.feat.yaml`)
- LinkML schemas (`def/`)
- Implementations (`src/`)
- Tests
- Snapshots (`tests/snapshots/`)

Projects SHOULD NOT version control:
- Generated code (`gen_def/`, `gen_int/`)
- Build artifacts

8c Feature Definition Validation

A partial feat file is valid. All sections are optional; absent sections produce no output. A feat is valid as long as every cross-reference it declares resolves within the same feat file. For example, a feat with only `commands`, `queries`, and `procedures` (but no `events`, `policies`, or `projections`) is valid provided the procedure's `command` and `queries` fields reference names declared in that same feat — no `emits` means no event references to validate.

The generator MUST validate cross-references before code generation:
- All `command` references in procedures exist in `commands:`
- All `event` references in policies and projections exist in `events:`
- All `model` references in queries exist in `models:`
- All `models` references in projections exist in `models:`
- All `queries` references in procedures exist in `queries:`
- All `emits` references in procedures exist in `events:`
- All `emits` references in policies exist in `commands:`

8d Code Generation Idempotency

Running code generation multiple times MUST produce identical output:
- `dizzy def` produces same stubs given the same Feature Definition (skips existing files)
- `dizzy gen` produces same protocols given the same feat file and `def/` files
- Timestamps and random IDs are never included in generated code

9 Deployment Specifications

9a Deployment Independence

Implementations in `src/` MUST NOT:
- Import deployment-specific libraries
- Reference environment-specific configuration
- Assume specific messaging infrastructure
- Hardcode database connection strings

9b Wiring Requirements

Deployment code MUST:
- Instantiate Context with concrete emitters
- Provide Query implementations backed by real SQLAlchemy sessions
- Handle serialization and deserialization
- Manage infrastructure connections

9c Deployment Patterns

9c1 Serverless (AWS Lambda)

```
def lambda_handler(event, context):
    command = parse_command(event)
    procedure_context = extract_and_transform_recipe_context(
        emit=extract_and_transform_recipe_emitters(
            recipe_ingested=lambda e: sqs.send_message(e),
        ),
        query=extract_and_transform_recipe_queries(
            get_recipe_text=make_get_recipe_text(dynamodb_session),
        ),
    )
    extract_and_transform_recipe(procedure_context, command)
```

9c2 Message Queue Worker

```
def worker_loop():
    while True:
        message = queue.receive()
        command = deserialize(message)
        procedure_context = create_context(
            emitters=queue_emitters(),
            queries=postgres_queries(),
        )
        procedure(procedure_context, command)
        queue.ack(message)
```

9c3 Monolithic Service

```
def handle_request(request):
    command = request_to_command(request)
    procedure_context = create_context(
        emitters=in_memory_emitters(),
        queries=sqlite_queries(),
    )
    procedure(procedure_context, command)
    return response_from_events()
```

9d Scalability

Deployments MAY:

- Run multiple instances of Procedures and Policies
- Partition the Event Store
- Shard Models across databases
- Use read replicas for Queries

Deployments MUST ensure:

- At-least-once Event delivery
- Idempotent Projection updates

- Event ordering within partitions

10 Security Considerations

10a Event Store Security

The Event Store:

- MUST be append-only (no deletions or modifications)
- MUST authenticate all write operations
- SHOULD encrypt Events at rest
- SHOULD encrypt Events in transit

10b Command Validation

Commands MUST:

- Be validated against schema before processing
- Include authentication/authorization context
- Reject Commands from untrusted sources

10c Query Authorization

Queries MUST:

- Enforce access control based on caller identity
- Prevent unauthorized data access
- Log query access for audit
- Rate limit to prevent DoS

10d Procedure Side Effects

Procedures that interact with external systems MUST:

- Validate all external inputs
- Use secure communication channels
- Handle credentials securely
- Implement retry with exponential backoff

10e Sensitive Data

Events containing sensitive data SHOULD:

- Be encrypted in the Event Store
- Use field-level encryption where appropriate
- Implement data retention policies
- Support GDPR/privacy compliance (e.g., event redaction)

10f Deployment Security

Deployment configurations MUST:

- Store credentials in secure vaults (not version control)
- Use least-privilege access for components
- Implement network isolation where appropriate

- Regularly update dependencies for security patches

11 References

11a Normative References

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997.

[LinkML] "Linked Data Modeling Language", https://linkml.io/

[CQRS] Fowler, M., "CQRS", https://martinfowler.com/bliki/CQRS.html

[EventSourcing] Fowler, M., "Event Sourcing", https://martinfowler.com/eaaDev/EventSourcing.html

11b Informative References

[DDD] Evans, E., "Domain-Driven Design: Tackling Complexity in the Heart of Software", Addison-Wesley, 2003.

[CosmicPython] Percival, H. and Gregory, B., "Architecture Patterns with Python", O'Reilly, 2020, https://www.cosmicpython.com/

[Kleppmann] Kleppmann, M., "Designing Data-Intensive Applications", O'Reilly, 2017.

12 Appendix A: Change Log

Version 0.2.0 (2026-03-12):

- Unified SPECIFICATION.md (v0.2, repo-authoritative) and specification.md (v0.1, older draft)
- Corrected CLI commands: `dizzy def` / `dizzy gen` (supersedes old `dizzy gen init` / `dizzy gen src`)
- Corrected directory layout: `gen_def/` + `gen_int/` (supersedes old `gen/pyd/`)
- Corrected commands and events schema layout: single `def/commands.yaml` and `def/events.yaml` (supersedes per-file `def/commands/{name}.yaml` pattern)
- Corrected naming conventions: `snake_case` for commands, events, models (supersedes PascalCase)
- Corrected policy context: emitters only — no query access for policies
- Added feat file specification (Section 5) with per-section schema, example, and generated output
- Added testing strategy (Section 7): render/write split, syrupy snapshots, Typer CLI tests
- Retained deployment patterns, security considerations, and data flow specs from v0.1
- Removed: `impl.yaml` implementation manifest, `result/` deployment artifacts directory

Version 0.1.0 (2026-01-31):

- Initial draft specification
- Core architecture definition
- Component specifications

- Code generation pipeline

—

End of Specification